1. INTRODUCTION

Reference ISA: The Mohloli ISA is a custom RISC-based instruction set architecture designed as the computational core for low-cost, AI-enabled mobile phones targeting the African market. It is a 32-bit load-store architecture that extends a simple base ISA with custom instructions to accelerate key localized AI workloads such as voice recognition and biometric authentication.

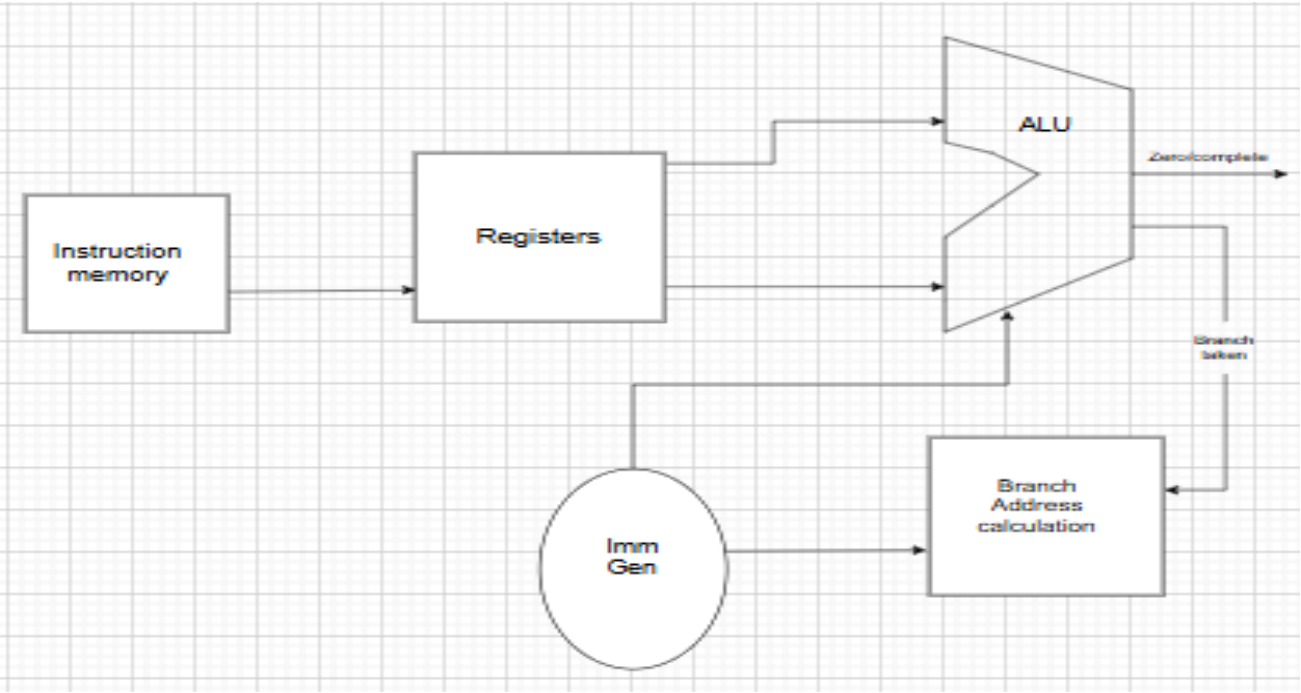
Supported Instruction Classes: The ISA supports six primary instruction classes:

- R-type: Integer and Floating-Point Arithmetic (plus, min, mult, fplus, fmin, fmult)
- I-type: Immediate Arithmetic and Loads (plusi, load, fload)
- S-type: Store Operations (store, fstore)
- SB-type: Conditional Branches (branchEQ, branchNE, branchGT)
- UJ-type: Jump and Link (jumpL)
- Custom R-type: Vector Operations (vecMAC, vecSubSq)
- Instruction fetch: reads the current instruction from the memory.

Design Objective: The main design goals for the microarchitecture are to enable a simple and efficient pipeline implementation, minimise power consumption for extended battery life, and provide high performance for the targeted AI workloads through the effective integration of the custom functional units, all while maintaining a low hardware cost.

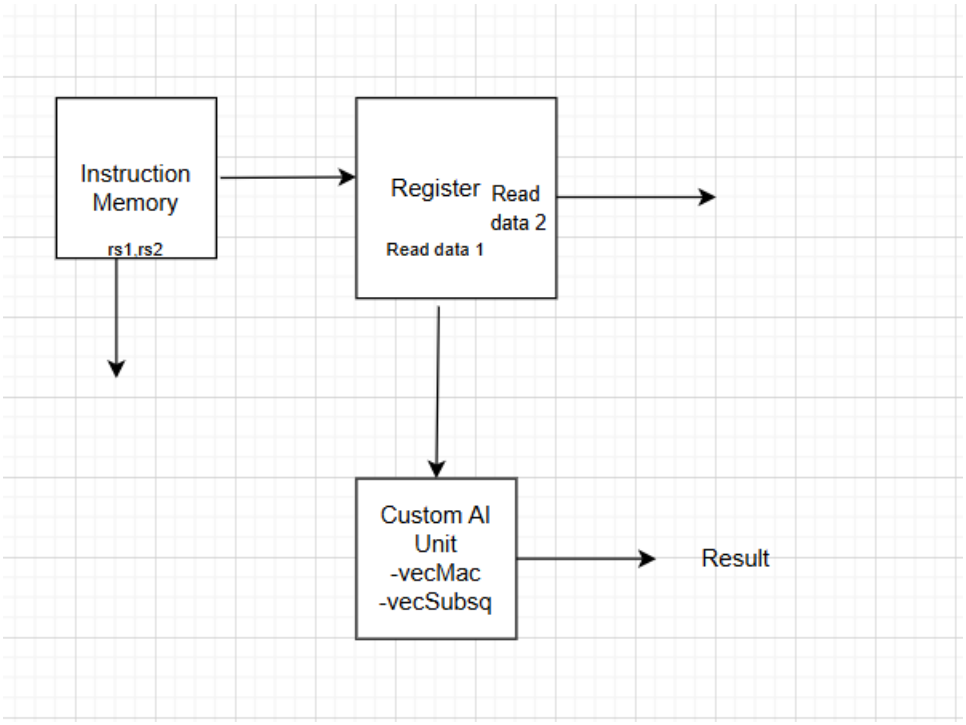
2.

Branch Instuctions:



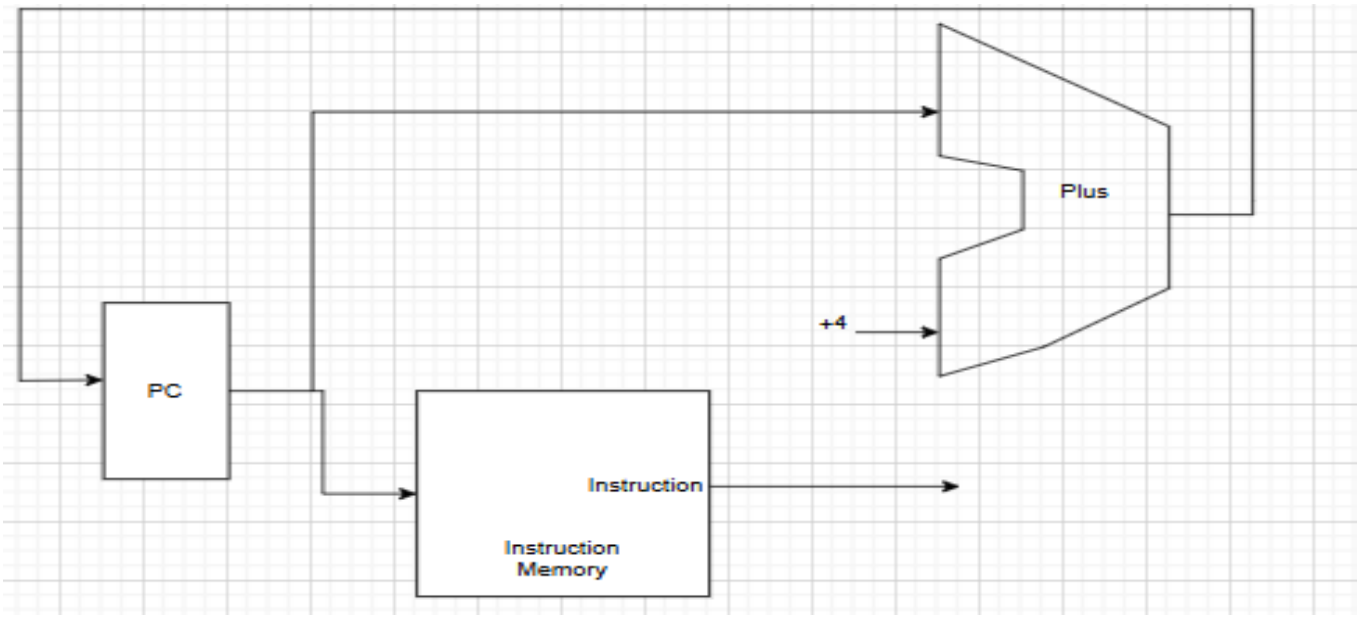
Conditional branching is handled through a coordinated process that evaluates register comparisons and calculates target addresses. The process begins when the Instruction Memory supplies a branch instruction (branchEQ, branchNE, or branchGT) to the pipeline, which then reads two source registers (rs1 and rs2) from the Register File. These register values are fed into the Arithmetic Logic Unit (ALU), which performs a comparison operation (typically subtraction) and generates a Zero/Complete status flag indicating whether the branch condition is satisfied. Simultaneously, the Immediate Generator (Imm Gen) extracts and sign-extends the 12-bit offset from the instruction, which is then used by the Branch Address Calculation unit to compute the target address by adding the offset to the current program counter. If the ALU's comparison result indicates the branch condition is met (branch taken), the processor updates the PC with the calculated branch target address, redirecting program flow; otherwise, execution continues sequentially with PC+4. This efficient branch mechanism enables our Mohloli processor to handle the complex control flow requirements of AI applications like decision-making in health monitoring and conditional processing in mobile financial services.

Custom AI instruction:



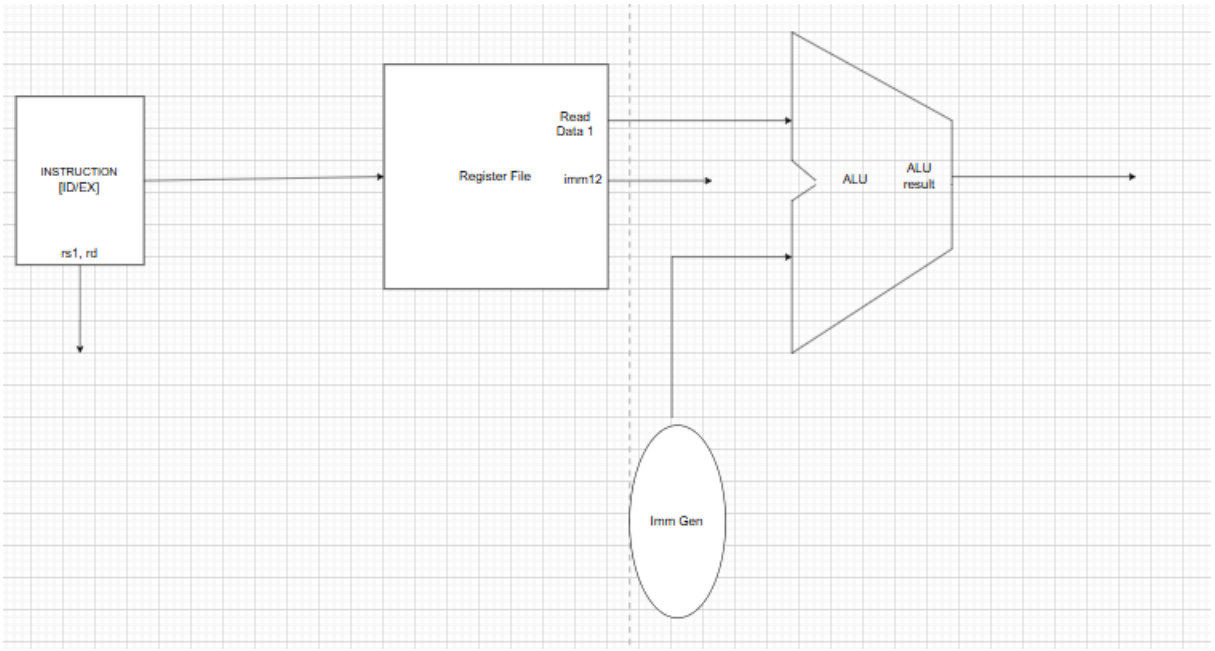
In the Instruction Fetch (IF) stage of our Mohloli processor implementation for the CS3520 lab, the execution begins with the Program Counter (PC) register holding the memory address of the current instruction to be executed. This address is fed into the Instruction Memory unit, which reads and outputs the corresponding 32-bit instruction word stored at that location. Simultaneously, the PC value is routed to an adder circuit that increments it by 4 (since we use 32-bit instructions in our Mohloli ISA), calculating the address of the next sequential instruction. This PC+4 value is then stored back into the PC register on the next clock cycle, ensuring the processor advances to the subsequent instruction in memory. This fetch mechanism forms the foundational first stage of our pipelined design, reliably supplying instructions to the subsequent decode stage while maintaining proper program flow through sequential execution or preparing for potential control flow changes from branch and jump instructions.

Instruction Fetch:



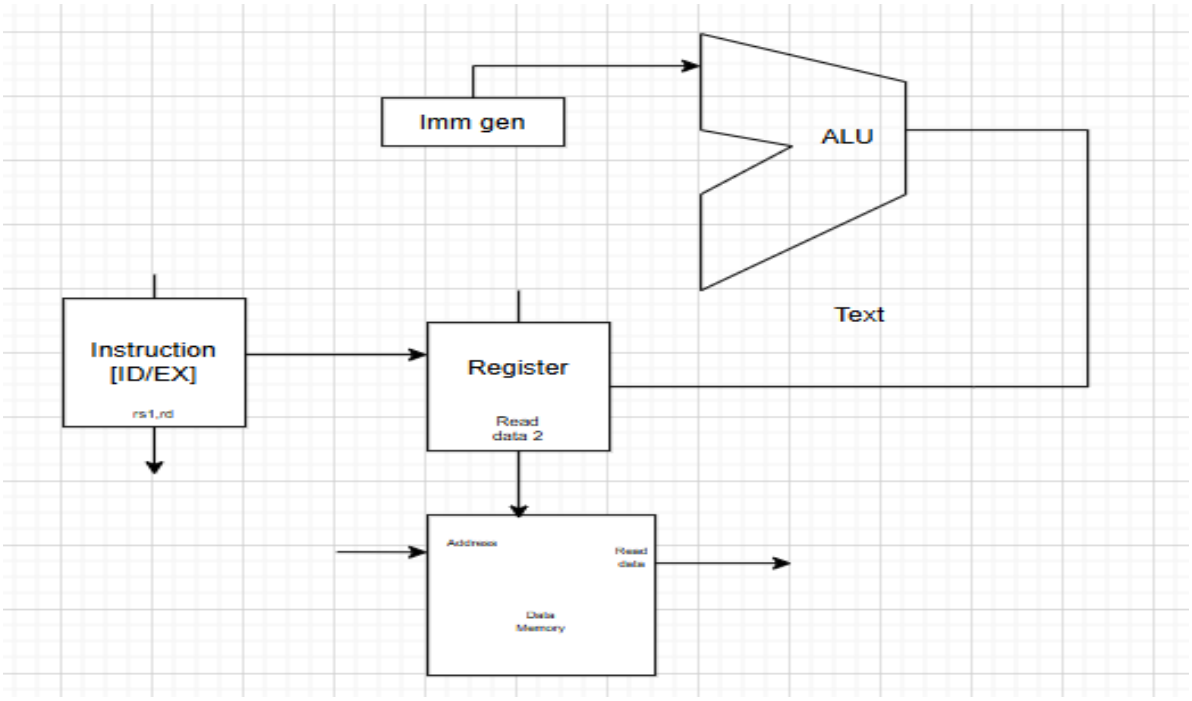
In the Instruction Fetch (IF) stage of our Mohloli processor implementation for the CS3520 lab, the execution begins with the Program Counter (PC) register holding the memory address of the current instruction to be executed. This address is fed into the Instruction Memory unit, which reads and outputs the corresponding 32-bit instruction word stored at that location. Simultaneously, the PC value is routed to an adder circuit that increments it by 4 (since we use 32-bit instructions in our Mohloli ISA), calculating the address of the next sequential instruction. This PC+4 value is then stored back into the PC register on the next clock cycle, ensuring the processor advances to the subsequent instruction in memory. This fetch mechanism forms the foundational first stage of our pipelined design, reliably supplying instructions to the subsequent decode stage while maintaining proper program flow through sequential execution or preparing for potential control flow changes from branch and jump instructions.

I-type Instruction:



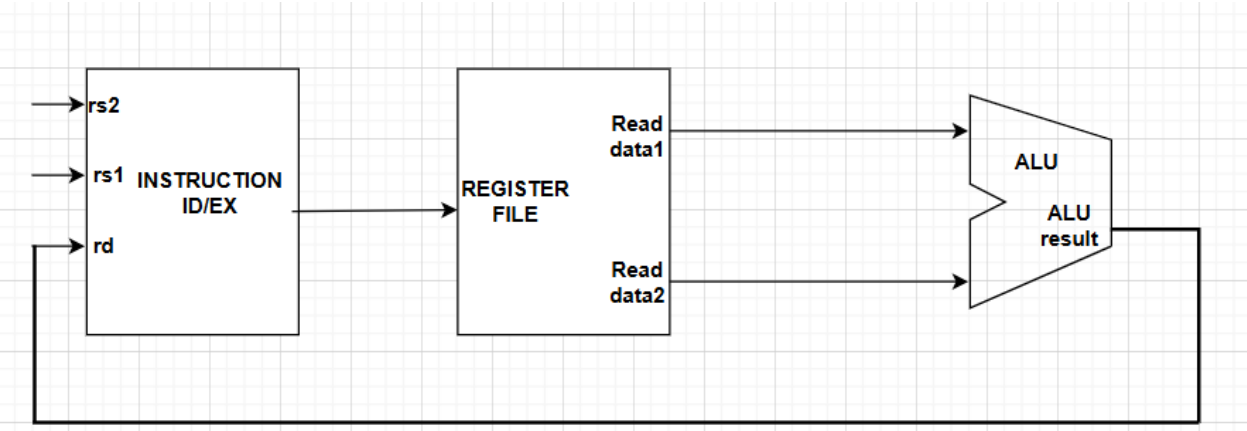
An **I-type instruction** executes by performing an arithmetic or logical operation using one register operand and an immediate value. When the instruction is fetched from memory, it is decoded to identify the source register (rs1), destination register (rd), the operation type (from the opcode and funct3 fields), and the 12-bit immediate value (imm12). The value stored in rs1 is read from the register file, while the immediate value is sign-extended by the Immediate Generator. Both operands — the register data and the immediate — are sent to the ALU, which performs the specified operation such as addition, subtraction, or bitwise logic. The result produced by the ALU is then written back to the destination register (rd). This allows the processor to efficiently perform computations using constants without needing to load them from memory.

Memory Instruction:



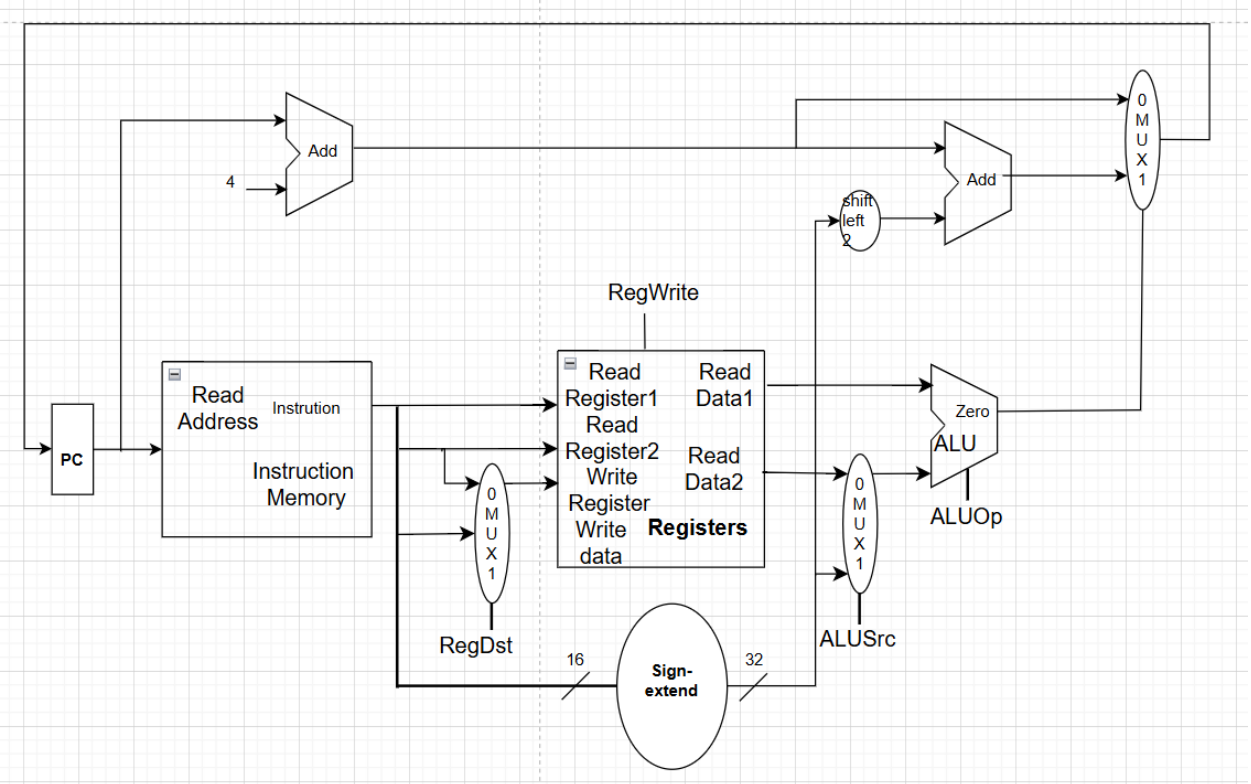
the system employs a structured approach to manage data access efficiently across different storage levels. The hierarchy begins with the Instruction Memory at the top, which stores the program code and feeds instructions to the pipeline during the fetch stage. For data operations, the system utilizes Data Memory accessed during the MEM stage, where addresses computed by the ALU are used to read or write operand values. The Register File acts as the fastest level of storage, providing immediate access to frequently used data during the ID stage through direct register addressing (rs1, rs2, rd). Between these levels, the Immediate Generator (Imm Gen) handles constant values embedded within instructions, while memory management units coordinate data flow between registers and main memory. This multi-tiered approach ensures that our processor maintains optimal performance for AI workloads like voice recognition and biometric processing by minimizing memory access latency while efficiently handling the data movement requirements of our custom Mohloli ISA instructions.

R-type Instruction:



For an R-type instruction, the fields rs1, rs2, and rd are decoded in the ID/EX stage. The **Register File** reads the contents of registers rs1 and rs2 and outputs them to the **ALU**. The ALU then performs the arithmetic or logical operation specified by the instruction (such as add, sub, and, or, etc.). The **ALU result** is then written back into the destination register rd in the register file.

J-type Instruction:

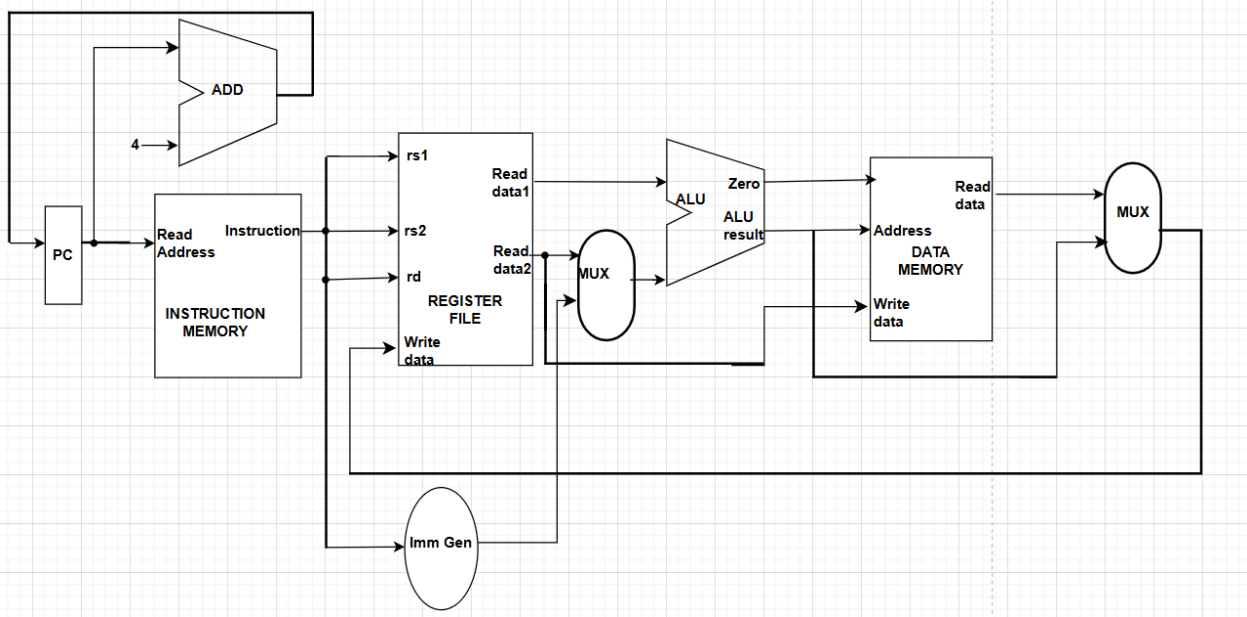


In a **J-type instruction** (Jump-type instruction), the execution process involves updating the **Program Counter (PC)** to a new address specified within the instruction, allowing an unconditional jump to a target location in memory. The instruction is fetched from the **Instruction Memory** using the current PC

value. The **opcode** is extracted to identify it as a J-type instruction, while the remaining 26 bits represent the **jump target address**. This address is shifted left by two bits (since instructions are word-aligned) and combined with the upper bits of the incremented PC (PC + 4) to form the full 32-bit jump address. Finally, this computed address is loaded back into the PC through a multiplexer, causing program execution to continue from the new target location. Unlike R-type or I-type formats, J-type instructions do not use registers or the ALU for computation.

In a **J-type instruction** (Jump-type instruction), the execution process involves updating the **Program Counter (PC)** to a new address specified within the instruction, allowing an unconditional jump to a target location in memory. The instruction is fetched from the **Instruction Memory** using the current PC value. The **opcode** is extracted to identify it as a J-type instruction, while the remaining 26 bits represent the **jump target address**. This address is shifted left by two bits (since instructions are word-aligned) and combined with the upper bits of the incremented PC (PC + 4) to form the full 32-bit jump address. Finally, this computed address is loaded back into the PC through a multiplexer, causing program execution to continue from the new target location. Unlike R-type or I-type formats, J-type instructions do not use registers or the ALU for computation.

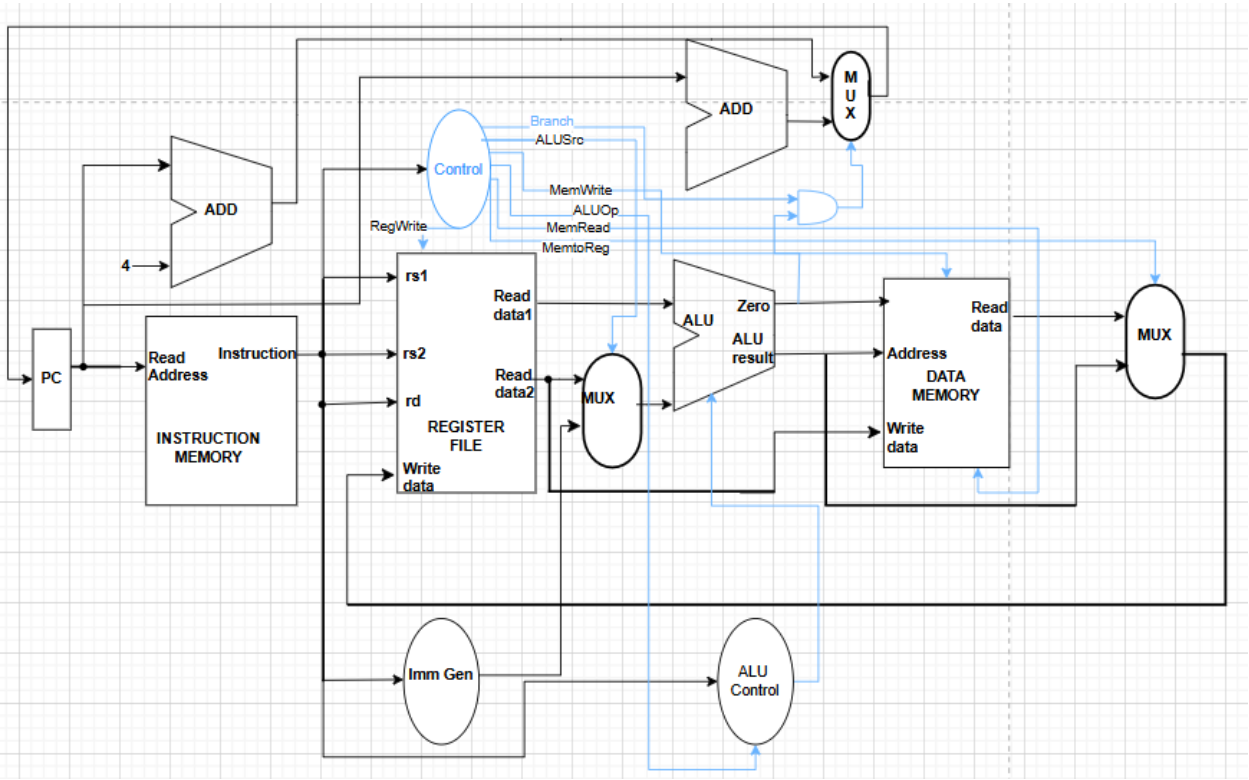
3.Combined Datapath:



Instruction execution flows sequentially through all hardware components in a coordinated manner. The process begins with the Program Counter (PC) supplying the instruction address to Instruction Memory, which fetches the 32-bit instruction while simultaneously calculating the next address via the adder. The fetched instruction then proceeds to the Register File where source registers (rs1, rs2) are read, providing operand data, while the Immediate Generator extracts and prepares any immediate values from the instruction. The execution phase routes these operands through multiplexers to the Arithmetic Logic Unit (ALU), which performs the specified computation and generates both result data and status flags like the Zero flag for branch decisions. For memory operations, the ALU result serves as an address input to Data Memory for load/store accesses, while the custom AI Unit stands ready to handle

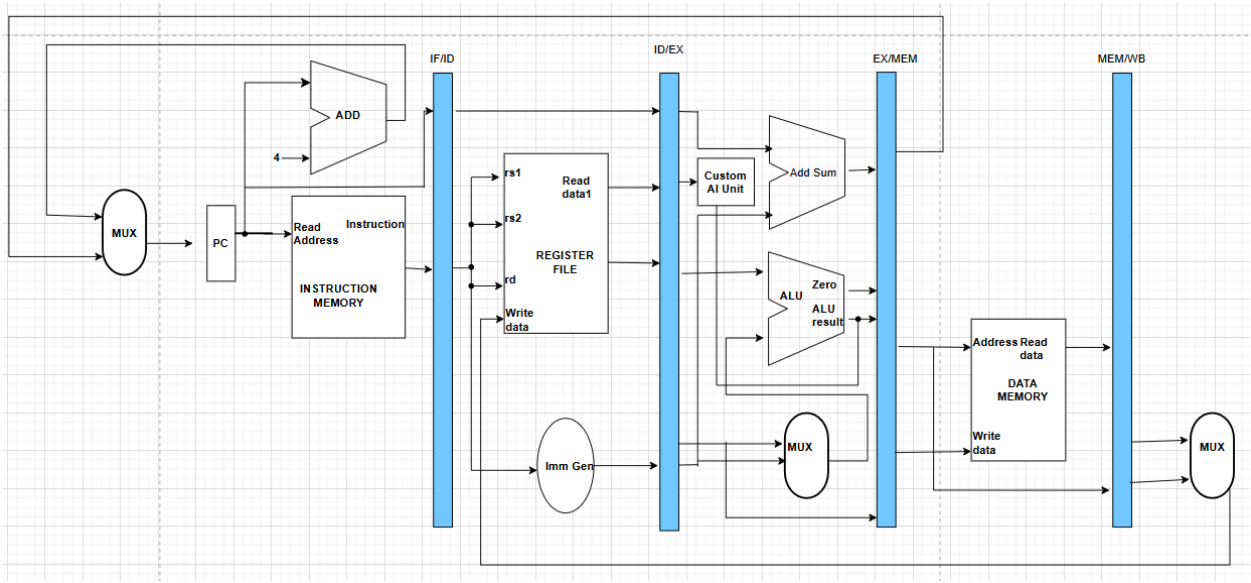
specialized vector operations like vecMAC and vecSubSq. Finally, the result selection multiplexer chooses between ALU outputs and memory read data, delivering the final value back to the Register File for writing to the destination register (rd), completing the instruction's journey through our comprehensive Mohloli datapath architecture.

4. Control Unit Design



Instruction	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp	VecOp
R-type	1	0	0	0	0	0	10	00
I-type (arith)	1	1	0	0	0	0	10	00
Load	1	1	1	0	1	0	00	00
Store	0	1	0	1	X	0	00	00
Branch	0	0	0	0	X	1	01	00
F-type	1	0	0	0	0	0	11	00
vecMAC	1	0	0	0	0	0	XX	01
vecSubSq	1	0	0	0	0	0	XX	10

5. PIPELINED IMPLEMENTATION



Hazard Resolution:

- Data Hazards: Solved through operand forwarding from later pipeline stages. When detected, the Forwarding Unit multiplexes the correct data to the ALU inputs.
- Control Hazards: For branches, uses predict-not-taken strategy. When a branch is taken, the pipeline flushes the two incorrectly fetched instructions (2-cycle penalty).
- Load-Use Hazards: Detected by Hazard Unit, which inserts a single pipeline stall (bubble) when a load instruction is followed by an instruction that uses the loaded data.

6. Memory Hierarchy Design

The Mohloli processor employs a two-level cache hierarchy optimized for low power and area:

- L1 Instruction Cache (I-Cache): 8KB, 2-way set associative
- L1 Data Cache (D-Cache): 8KB, 2-way set associative
- Unified L2 Cache: 128KB shared (off-chip)
- Main Memory: LPDDR4 SDRAM

The separate I/D L1 caches (Harvard architecture) allow simultaneous instruction and data access, improving throughput for the target AI workloads that often involve streaming data processing alongside control code execution.

7. **CONCLUSION**

The Mohloli microarchitecture successfully balances the competing demands of computational performance, power efficiency, and implementation simplicity. By extending a classic RISC pipeline with domain-specific functional units and employing careful hazard management, it delivers accelerated performance on target AI workloads while maintaining the low-cost profile essential for the African

mobile market. The design demonstrates how thoughtful micro architectural choices can tailor general-purpose processors to specific application domains without excessive complexity.

REFERENCES

1. Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann. (The standard textbook for computer architecture courses)
2. Waterman, A., & Asanović, K. (Eds.). (2019). The RISC-V Instruction Set Manual, Volume I: User-Level ISA (Document Version 20191213). RISC-V Foundation. (The official specification for the RISC-V ISA, which your project is based on)
3. Harris, D., & Harris, S. (2016). Digital Design and Computer Architecture: RISC-V Edition. Morgan Kaufmann. (A more modern textbook focusing specifically on RISC-V)
4. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann. (The advanced companion to "Computer Organization and Design")
5. National University of Lesotho. (2025). CS3520 - Computer Organisation and Architecture I: Project Handout. Department of Mathematics and Computer Science.